Software Testing Life Cycle (STLC)

1. Introduction

Software testing is not simply about executing test cases and finding bugs after developers finish writing code. It is a systematic process that begins with understanding requirements and continues through test planning, test design, environment preparation, execution, and test closure.

To organize these activities effectively, software testing teams follow a structured process known as the **Software Testing Life Cycle (STLC)**.

STLC defines the different phases involved in software testing, the activities performed during each phase, and the deliverables produced by the testing team.

2. Definition of STLC

The **Software Testing Life Cycle (STLC)** is a systematic process followed by software testing teams to plan, analyze, design, prepare, execute, and complete testing activities to verify that a software application meets its specified requirements and quality expectations.

Each phase has specific objectives, activities, entry criteria, exit criteria, and deliverables.

The primary objective of STLC is to ensure that software quality is evaluated systematically and that defects are identified and communicated effectively before the software is released.

3. Why Do We Need STLC?

Consider an online shopping application that supports registration, login, product browsing, cart management, and payments.

If testers start testing without understanding the requirements or preparing test cases, important scenarios may be missed. Testing might become unorganized, defects may not be tracked properly, and the team may not know whether sufficient testing has been completed.

STLC provides a structured approach to avoid these problems.

Its major purposes include:

- Understanding what needs to be tested.
- Identifying testing scope and objectives.
- Estimating testing effort and resources.
- Designing effective test scenarios and test cases.
- Preparing suitable test environments and test data.
- Executing tests systematically.
- Identifying, reporting, and tracking defects.
- Evaluating testing progress and completion status.

4. Phases of STLC

STLC is commonly explained through six major phases.

4.1 Requirement Analysis

This is the first phase of STLC. During this phase, the testing team studies the available business and software requirements to understand what the application should do and what needs to be tested.

Testers identify testable requirements and clarify incomplete, ambiguous, or conflicting requirements with stakeholders.

Activities:

- Reviewing business and functional requirements.
- Understanding application functionality.
- Identifying testable requirements.
- Identifying positive, negative, and boundary scenarios.
- Clarifying doubts with Business Analysts and developers.
- Identifying testing types required for the application.
- Evaluating automation feasibility.

Deliverables:

- Requirement analysis document.
- List of testable requirements.
- Requirement clarification records.
- Initial Requirement Traceability Matrix (RTM).

Example:

Suppose the requirement states:

"A user account must be locked for 15 minutes after five consecutive failed login attempts."

A tester analyzes this requirement and identifies scenarios such as:

- Login with valid credentials.
- Login with an incorrect password.
- Enter an incorrect password five consecutive times.
- Attempt login immediately after account lockout.
- Attempt login after the 15-minute lockout period.

The purpose is to understand the expected behavior before designing the test cases.

4.2 Test Planning

After understanding the requirements, the testing team prepares a plan describing how testing activities will be performed.

The Test Manager or Test Lead generally coordinates this phase, with input from the testing team.

Activities:

- Defining testing scope and objectives.
- Selecting testing strategies and approaches.
- Identifying testing types and levels.
- Estimating testing effort and duration.
- Identifying required resources.

- Selecting testing tools.
- Identifying risks and mitigation strategies.
- Defining roles and responsibilities.
- Establishing entry and exit criteria.

Deliverables:

- Test Plan.
- Test strategy or testing approach.
- Effort and schedule estimates.
- Resource allocation.
- Risk assessment.

Example:

For the ShopEasy application, the testing team may decide to perform functional testing for registration, login, product browsing, cart operations, and checkout.

The team may use Jira for test and defect management and Xray for organizing test cases and recording execution results.

4.3 Test Case Development

Once the testing approach is established, testers design detailed test cases based on the analyzed requirements.

A Test Case defines the conditions, actions, input data, and expected results required to verify a particular functionality.

Activities:

- Identifying test scenarios.
- Writing positive test cases.
- Writing negative test cases.
- Designing boundary-value scenarios.
- Preparing test data.
- Reviewing test cases.
- Mapping test cases to requirements.
- Identifying test cases suitable for automation.

Deliverables:

- Test Scenarios.
- Test Cases.
- Test Data.
- Updated Requirement Traceability Matrix.
- Test Case Review Records.

Example:

For the account lockout requirement, a test case may contain:

Field	Details
Test Case ID	TC-LOGIN-005
Scenario	Verify account lockout after five failed login attempts
Preconditions	A registered user account exists.
Test Data	Valid username and incorrect password
Steps	Attempt login five consecutive times using an incorrect password.
Expected Result	The account should be locked for 15 minutes.

A well-designed test case should be clear, repeatable, and capable of producing a verifiable result.

4.4 Test Environment Setup

A test environment is the configured infrastructure in which the application is tested.

It may include application builds, servers, databases, browsers, operating systems, configurations, network settings, and test accounts.

During this phase, the team ensures that the required environment is available and ready for testing.

Activities:

- Configuring test servers.
- Deploying the application build.
- Setting up databases.
- Installing required software and browsers.
- Preparing test accounts and test data.
- Configuring application settings.
- Verifying environment connectivity.
- Performing smoke testing to confirm basic application availability.

Deliverables:

- Configured test environment.
- Environment details and configuration records.
- Test data.
- Environment readiness or smoke-test results.

Example:

For ShopEasy, the testing team may prepare the following environment:

Parameter	Configuration
Application	ShopEasy
Build Version	1.0.3
Environment	Staging

Browser	Chrome – Latest
Operating System	Windows 11
Test Data	Sample customer accounts and products

The environment must be available and stable before meaningful test execution begins.

4.5 Test Execution

This is the phase in which testers execute the prepared test cases against the application and compare actual behavior with expected results.

The objective is to verify whether the application behaves according to the specified requirements.

Activities:

- Executing test cases.
- Entering required test data.
- Comparing actual results with expected results.
- Marking test cases as Pass, Fail, Blocked, or other supported execution statuses.
- Reporting defects for unexpected behavior.
- Retesting fixed defects.
- Performing regression testing.
- Updating execution progress.

Deliverables:

- Test Execution Results.
- Defect Reports.
- Retest Results.
- Updated Requirement Traceability Matrix.
- Execution progress reports.

Example:

A tester executes the account lockout test case and observes that the application unlocks the account after five minutes instead of the required 15 minutes.

The test case is marked as Failed, and a Bug is created with the following details:

- Expected Result: The account should remain locked for 15 minutes.
- Actual Result: The account becomes accessible after five minutes.
- Defect: Account lockout duration is shorter than required.

The defect is then assigned to the appropriate development team member for investigation and resolution.

4.6 Test Closure

Test Closure is the final phase of STLC. During this phase, the testing team evaluates the overall testing activities and prepares a summary of the results.

The team verifies whether the planned testing objectives have been achieved and documents any remaining risks or limitations.

Activities:

- Reviewing executed test cases.
- Calculating Pass and Fail percentages.
- Reviewing open and closed defects.
- Evaluating requirement coverage.
- Preparing the Test Summary Report.
- Documenting lessons learned.
- Archiving test artifacts.
- Obtaining testing sign-off where required.

Deliverables:

- Test Summary Report.
- Final execution metrics.
- Defect summary.
- Requirement coverage information.
- Lessons learned.
- Test closure or sign-off records.

Example:

Suppose a testing cycle contains 100 Test Cases:

Execution Status	**Count**
Passed	85
Failed	10
Blocked	3
Not Run	2
Total	**100**

The testing team reviews these results, evaluates the outstanding defects, identifies release risks, and communicates whether the application is ready for release based on the agreed exit criteria.

5. STLC Workflow

The following diagram illustrates the commonly used sequential representation of STLC.

Requirement Analysis

|

v

Test Planning

|

v

Test Case Development

|

v

Test Environment Setup

|

v

Test Execution

|

v

Test Closure

The phases are connected, but they do not always operate as strictly isolated steps. Activities may overlap, repeat, or be revisited when requirements change or defects are discovered.

6. Important Testing Terminology

Term	**Definition**
Test Scenario	A high-level description of what functionality needs to be tested.
Test Case	A set of preconditions, steps, inputs, and expected results used to verify a specific behavior.
Test Data	Input values used while executing test cases.
Test Plan	A document describing the testing scope, objectives, strategy, resources, schedule, and approach.
Test Environment	The configured setup in which testing is performed.
Actual Result	The behavior or output observed after executing a test case.
Expected Result	The behavior or output that should occur according to the requirement.
Defect / Bug	A flaw that causes the software to behave differently from its expected behavior.
Retesting	Testing a previously failed scenario again after the related defect has been fixed.
Regression Testing	Testing existing functionality to ensure that changes have not introduced unintended defects.
RTM	Requirement Traceability Matrix, used to map requirements to their corresponding test cases and execution coverage.

7. Difference Between Retesting and Regression Testing

These two concepts are important during the Test Execution phase.

Retesting	**Regression Testing**
Verifies whether a previously reported defect has been fixed.	Verifies whether existing functionality still works after changes.
Focuses on the specific failed scenario.	May cover multiple related and unaffected areas.
Uses the previously failed test scenario.	Uses relevant existing test cases.
Example: Checking whether the account now remains locked for 15 minutes after a fix.	Example: Checking whether login, logout, and password reset still work after changes to authentication logic.

8. Relationship Between SDLC and STLC

SDLC and STLC are closely related processes that work together to deliver quality software.

SDLC covers the complete software development process, while STLC focuses on organizing and performing testing activities.

SDLC	**STLC**
Focuses on software development.	Focuses on software testing.
Covers requirements, design, coding, testing, deployment, and maintenance.	Covers requirement analysis, test planning, test development, environment setup, execution, and closure.
Involves developers, architects, testers, and other stakeholders.	Primarily involves QA engineers, testers, test leads, and supporting stakeholders.
Produces software and related development deliverables.	Produces test plans, test cases, execution results, defect reports, and test summaries.

Important: Testing is not limited to the final testing phase of SDLC. Testers can participate during requirement analysis and design reviews, helping identify defects and ambiguities early.

9. Responsibilities of an SDET During STLC

A Software Development Engineer in Test (SDET) contributes to testing activities while also applying software engineering skills.

Depending on the project and team structure, an SDET may participate in:

- Reviewing requirements and identifying test scenarios.
- Designing and developing test cases.
- Writing automation scripts.
- Developing and maintaining test automation frameworks.
- Preparing test data and environments.
- Executing manual and automated tests.
- Investigating failures and analyzing logs.
- Reporting and tracking defects.
- Performing retesting and regression testing.
- Integrating automated tests into CI/CD pipelines.

- Contributing to test reports and quality metrics.

An SDET does not simply execute test cases. The role also involves building software-based solutions that improve the efficiency, reliability, and scalability of testing.

10. Conclusion

The Software Testing Life Cycle provides a structured approach to planning, designing, preparing, executing, and completing software testing activities.

By following STLC, testing teams can improve test coverage, identify defects systematically, maintain traceability between requirements and tests, and evaluate software quality before release.

Understanding STLC is essential for manual testers, automation testers, and SDETs because it explains how testing activities fit into the overall software development process.